# ADVERSARIAL ATTACKS ON TRANSFORMER BASED MALWARE DETECTOR

**A Project Report**

*Submitted by*


| | |
|---|---|
| **Heramb Patil** | **111603021** |
| **Jugal Rawlani** | **111603053** |
| **Yash Jakhotiya** | **111603067** |


*in partial fulfillment for the award of the degree*

*of*

**B.Tech Computer Engineering**

Under the guidance of

**Dr. Sunil B. Mane**

College of Engineering, Pune


**DEPARTMENT OF COMPUTER ENGINEERING**

**AND**

**INFORMATION TECHNOLOGY,**

**COLLEGE OF ENGINEERING, PUNE-5**


September, 2020

# DEPARTMENT OF COMPUTER ENGINEERING

# AND

# INFORMATION TECHNOLOGY,

# COLLEGE OF ENGINEERING, PUNE

## CERTIFICATE

Certified that this project, titled "ADVERSARIAL ATTACKS ON TRANSFORMER BASED MALWARE DETECTOR" has been successfully completed by

| | |
|---|---|
| **Heramb Patil** | **111603021** |
| **Jugal Rawlani** | **111603053** |
| **Yash Jakhotiya** | **111603067** |

and is approved for the partial fulfillment of the requirements for the degree of "B.Tech. Computer Engineering".

SIGNATURE
**Dr. S. B. Mane**
**Project Guide**
**Department of Computer Engineering and Information Technology,**
**College of Engineering Pune,**
**Shivajinagar, Pune - 5.**

SIGNATURE
**Dr. V. Z. Attar**
**Head**
**Department of Computer Engineering and Information Technology,**
**College of Engineering Pune,**
**Shivajinagar, Pune - 5.**

# Abstract

Malware or malicious software are programs designed to steal user credentials, corrupt data, encrypt files for ransom or do any of the possible nefarious tasks that can be done to damage a victim computer or network.

Signature-based malware detectors have proven to be unsuccessful as even a small change in malignant executable code can bypass these signature-based detectors. Many machine learning-based models have been proposed to efficiently detect a wide variety of malware. Many of these models are found to be susceptible to adversarial attacks - attacks that work by generating intentionally designed inputs that can force these models to misclassify. Our work aims to explore vulnerabilities in the current state of the art transformer-based malware detectors to adversarial attacks.

# Contents

**List of Figures** **ii**

**1 Introduction** **1**

1.1 Malware . . . . . 1

1.2 Malware Detectors . . . . . 2

1.3 Machine Learning in Malware Detectors . . . . . 2

**2 Literature Survey** **5**

2.1 Deep Neural Network-based techniques for Malware Detection 5

2.1.1 Proposed Method . . . . . 5

2.2 Adversarial attacks on Deep Neural Network based Malware Detectors . . . . . 7

2.2.1 Proposed Solution . . . . . 7

2.2.2 Conclusion . . . . . 9

2.3 CNN based Neural Networks for Malware Detection . . . . . 11

2.3.1 Proposed Solution . . . . . 11

2.3.2 Conclusion . . . . . 12

2.4 Adversarial attacks on CNN based Malware Detectors . . . . . 13

2.4.1 Proposed Solution . . . . . 13

2.4.2 Conclusion . . . . . 15

2.5 RNN based Neural Networks for Malware Detection . . . . . 16

2.5.1 Proposed Solution . . . . . 16

2.5.2 Conclusion . . . . . 16
2.6 Adversarial attacks on RNN based Malware Detector . . . . . 17
2.6.1 Proposed Solution . . . . . 18
2.7 Transformer based Neural Networks for Malware Detection . . 19
2.7.1 Proposed Solution . . . . . 19
2.7.2 Conclusion . . . . . 22

**3 Problem Statement and Objectives 23**
3.1 Problem Statement . . . . . 23
3.2 Objectives . . . . . 23

**4 Systems Requirement Specification 24**
4.1 Functional Requirements . . . . . 24
4.2 Non-Functional Requirements . . . . . 24
4.3 Hardware Requirements . . . . . 25
4.4 Software Requierments . . . . . 25

**5 System Design and Architecture 26**

**6 Implementation and Results 28**
6.1 Dataset Preparation . . . . . 28
6.2 Malware Detection Model . . . . . 29
6.2.1 Transformer Architecture . . . . . 30
6.2.2 Static Features . . . . . 32
6.2.3 Feature Set . . . . . 32
6.3 Adversarial attacks and Defenses . . . . . 33
6.3.1 Introduction . . . . . 33
6.3.2 Classification of attacks . . . . . 33
6.3.3 Our Approach to the attack . . . . . 35

6.3.4 Results . . . . . 36
6.3.5 Defenses against adversarial attacks . . . . . 36

**7 Conclusion and Future Scope** **38**

# List of Figures


2.1 High level architecture of deep neural network model for malware detection . . . . . 6

2.2 DNN Classifier . . . . . 8

2.3 Enhanced Appending Benign Feature Model Architecture . . . 15

2.4 Training of greyscale image genereated using CNN . . . . . 17

2.5 Generative recurrent neural network algorithm . . . . . 18

2.6 Overview of Interpretable feed-forward neural network . . . . 21

2.7 I-MAD model interpretation . . . . . 22

5.1 System Architecture . . . . . 26

6.1 Transformer Architecture . . . . . 30

6.2 Encoder . . . . . 31

# Chapter 1

# Introduction

## 1.1 Malware

MALWARE is software written to steal credentials of computer users, damage computer systems, and encrypt documents for ransom, among other nefarious goals. Since the Internet has become an integral part of people's life, the large volume of malware spreading on it imposes tremendous threats to billions of netizens. According to the McAfee Aug-2019 lab threat report, around 65 million new malwares was detected, and 1.25 million new ransomwares were detected, summing the total malware detected in 2019 to more than 900 million and these figures keep growing constantly[1].

Recognizing malware samples downloaded by legitimate users in a timely manner is thus of crucial importance for their protection. One of the major problems faced by the malware detectors community is detecting the sheer number of malwares that has been newly created and not been analysed before. Signature-based malware detection methods are widely used in antivirus products. With the signatures extracted by malware analysts, known malware samples or some of their variants can be precisely recognized. However, with obfuscation techniques or even a change of compiler, it is easy to create variants of known malware that perform the

same attack, but with literally different executable code. As a result, the previously crafted signatures can no longer recognize them. Furthermore, signature-based detection is also ineffective to detect new and unseen malware in most cases. There is thus a pressing need to create an intelligent malware detection system that can capture new malware or variants of known malware.

## 1.2 Malware Detectors

Signature-based malware detection methods are widely used in antivirus products. With the signatures extracted by malware analysts, known malware samples or some of their variants can be precisely recognized. However, with obfuscation techniques or even a change of compiler, it is easy to create variants of known malware that perform the same attack, but with literally different executable code. As a result, the previously crafted signatures can no longer recognize them. Furthermore, signature-based detection is also ineffective to detect new and unseen malware in most cases. There is thus a pressing need to create an intelligent malware detection system that can capture new malware or variants of known malware. For this purpose, many machine learning-based malware analysis methods have been proposed.

## 1.3 Machine Learning in Malware Detectors

Machine Learning models have several advantages over signature-based methods. First, they can automatically learn common patterns of malware from the feature space that have better generalization ability than manually crafted signatures. Second, machine learning models do not classify samples

based on the existence of individual signatures, but on a comprehensive analysis of the combination of patterns they identify from the feature space. Third, machine learning models do not require human experts to examine individual malware samples. They thus extensively save human labour.

Nonetheless, these neural network architectures harbour two serious limitations. One is their lack of interpretability. Linear models, such as logistic regression and Naive Bayes, are very simple and have excellent interpretability, but have inferior classification performance. The machine learning models that achieve state-of-the-art classification performance are therefore not linear models. The other limitation is that existing machine learning-based methods for malware detection are not based on understanding the semantic meaning of the assembly code of an executable.

In Sept 2017, Google introduced the Transformer, a self-attention mechanism-based neural network architecture that is believed to be pretty useful for different applications of language understanding. In their "Attention is all you need" paper, they show that this architecture performs better for both convolutional as well as recurrent models on academic English to French and English to German translation benchmarks. In addition to the greater performance on translation, the Transformer requires less computational runs to train and can be considered better support for the underlying machine learning hardware, along with increasing training speeds by many folds[2]. Due to similar semantic structures present in natural languages and assembly languages, Transformer-based malware detectors have recently been proposed to overcome the above two major limitations of previous neural network architectures for malware detection.

Deep learning is being used for solving a variety of complex problems and it

has provided a compact framework for dealing with problems that were difficult to solve using traditional machine learning techniques. In recent years, the advancement in deep learning has enabled it to provide performance at par with what humans can do on several tasks. As a result, deep learning is being used heavily in most of the recent day-to-day applications. On the other hand, deep learning systems are found to be vulnerable to different generated adversarial attacks, which may not be viewed or analysed by the human eye but can confuse the model to wrongly classify the output. Recently, several researchers have been successful in generating different kinds of adversarial attacks by leveraging vulnerabilities after analysing the threat model to eventually compromise the model.

# Chapter 2

# Literature Survey

## 2.1 Deep Neural Network-based techniques for Malware Detection

Malware continues to remain one of the most effective tools to compromise systems. And hence, malware needs to detected and purged before the malware could achieve its end goal. Most of the existing techniques used today for malware detection either use a rule-based approach or a signature-based approach. These approaches require a cybersecurity researcher to manually set up rules, or categorize a binary as malware and mark its signature. This would work if the researchers knew how malware works and is not a scalable approach. [17] Saxe et al propose a deep learning based approach to help solve this problem.

### 2.1.1 Proposed Method

- **Features**

    1. Byte Histogram Feature - Sliding a window of size 1024 bytes with 256 step size, and maintaining a count of each byte in the window to form a list, a 2D histogram is then plotted against the pair list, outputting 8 rows, [17] Saxe et al then concatenates the rows to

form a single vector, this feature helps in categorizing compressed malicious binaries.

2. PE Import Features - The authors extracted PE Imports from the import address table of the binary program and convert them to a vector representing imports in a particular file.
3. Strings - All the printable strings (ASCII characters only) of length greater than or equal to 6 are extracted from the binary file and converted to a feature vector.
4. PE Metadata features - PE metadata features are extracted and are compiled in a 256 length array to form a feature vector.

- **Dataset -** The dataset was compiled from sources such as VirusTotal, normal windows installation, etc. and contains 431,926 binaries, out of which 81,910 are benign and 350,016 are malware.

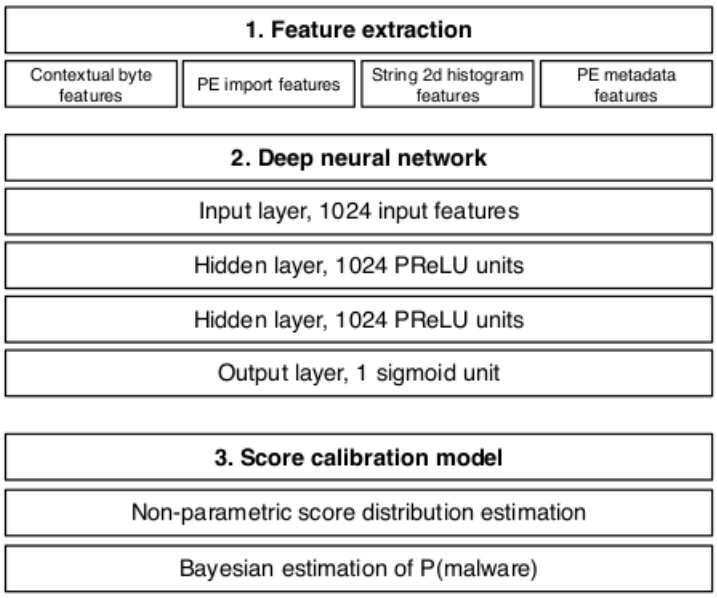


Figure 2.1: High level architecture of deep neural network model for malware detection

## 2.2 Adversarial attacks on Deep Neural Network based Malware Detectors

Recently, a lot of work has been focused around using Deep Learning for Malware Detection systems. But author, [18] Stokes et al describes this as a double-edged sword, where deep learning could be really helpful in identifying new, yet unknown malware, but miscreants can also come up with ways to fool the Neural Networks by creating something known as an Adversarial sample. [18] An Adversarial sample is a sample with small perturbations which does not change the sample's original function, but rather fools the network into classifying it into some other class.

### 2.2.1 Proposed Solution

- **Data Collection -** The dataset was collected from a clean installation of Windows, where each PE Executable file was scanned by an anti-malware engine. The dataset consisted of 2 major features, all printable strings from the executable, and all the system call the executable makes. These two combined form a feature vector of size 50,000 features.

- **DNN Classifier**The DNN model consists of 4 hidden layers, each comprising of 2000 nodes, and using ReLU as an activation function with a dropout rate of 25%, the final layer has only one node, and uses Softmax as the activation function.

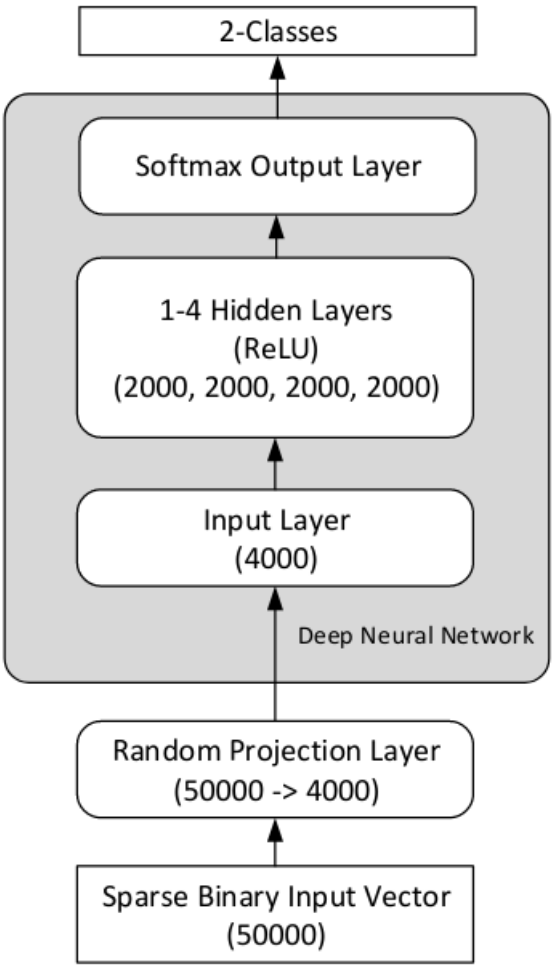


Figure 2.2: DNN Classifier

- **Adversarial Samples -** The authors assumed that the attacker has access to the feature vector, and the trained DNN model used for classifying the file as malicious or benign. [18] Stokes et al proposed 6 iterative strategies to instill perturbations in the malicious file to create an adversarial sample. But before that, instilling perturbations, the attacker needs to know the feature that influences the prediction the most. This can be done by calculating the Jacobian of the features. Stokes et al described positive features as the features that influence the prediction the most, and negative features as the features that influence the prediction the least.

1. Dec_pos : removing the features that would result in DNN based classifier to classify it as malicious.
2. Inc_neg: increasing the features that would result in DNN based classifier to classify it as benign.
3. Dec_pos + inc_neg: alternatively disabling one positive feature and enabling one negative feature to increase the probability of misclassification.
4.

- **Defensive Techniques -** The authors proposed 2 defensive mechanisms to prevent an adversarial attack on DNN based malware detectors.

  1. Ensemble Defense - Ensemble defense is basically training a number of neural networks and then use a majority voting to classify an executable as malicious or benign. The idea behind this is simple, that it is quite easy to fool one Neural network by adding some perturbations, but to fool a set of neural networks, it is not that easy.
  2. Weight Decay - Weight Decay is useful because when employed, large changes in the input result in lower changes in output when compared to a NN without Weight Decay.

### 2.2.2 Conclusion

The authors first evaluate the performance of the model before and after implementing the defenses

1. Base Model - For the baseline classifier the authors proposed, they were able to achieve a minimum of 1.13% Test Error Rate.

2. Weight Decay - After implementing weight decay method with different decay strengths, the model performed best with adversarial samples when decay strength was 0.0001.

3. Ensemble Defense - Authors tried training 3 and 5 models for ensemble defense, and observed that the possibility of an adversarial attack on the model decreased with increasing number of models.

## 2.3 CNN based Neural Networks for Malware Detection

The number of malware attacks has significantly increased over the past few decades, this could be attributed to the rise of the internet era. Most of the malware today are just some modification of previously found malwares, or differently obfuscated existing malwares. In this paper, the authors [19] Kalash et al have opted for a different approach of using CNN to classify binaries as malwares, unlike traditional methods of training vanilla neural nets for classification of malwares.

### 2.3.1 Proposed Solution

The authors proposed a CNN based approach to classify malwares. Malware as an image representation was used. A vector of 8 bits is first read from the given malware file. Each of this vector is converted to an unsigned integer in the range [0, 255]. This process is repeated for the complete malware binary, and we get a decimals for all corresponding vectors of 8 bits. This final vector of decimal is then reshaped to form a 2D matrix. The matrix is then visualized as a grayscale image. It is observed that similar malwares have similar textures when represented as a grayscale image.

**Model -** Stokes et al [18] proposed a CNN model based on VGG-16 architecture for classification purposes. The model takes a malware binary as input, and outputs the classification for the given malware file. The output class with maximum score is the prediction. The model uses cross-entropy loss for training, and parameters are learned using Stochastic Gradient Descent method.

### 2.3.2 Conclusion

The authors used two different datasets to train and test the model.

- **Malimg Dataset -** The malimg dataset consists of 9337 malware samples. These samples are then converted to grayscale images, and the model is trained. The authors used 90% of the data for training i.e. 8403 samples for training and the remaining 10% for testing i.e 934 samples for testing.

- **Microsoft Malware Dataset -** The Microsoft Malware Dataset consists of 21730 malware samples. Out of the given samples, 10845 are used for training the model, and the rest 10885 samples are used for testing purpose.

The authors were able to achieve best accuracy of 98.52% for Malimg dataset, and the best accuracy of 98.99% for Microsoft Malware Dataset.

## 2.4 Adversarial attacks on CNN based Malware Detectors

Over the last few decades, the number of malware attacks has risen dramatically, which may be due to the growth of the internet era. Much of today's malware is either an alteration to newly uncovered malware, or else obfuscated current malware. Previously, various authors have proposed a different approach of using CNN to identify binaries as malware, compared to conventional approaches of training vanilla neural networks for malware detection. However, as we know that CNNs are also vulnerable to adversarial attacks, and hence in this paper the authors: [20] Chen et al proposes various methods of conducting adversarial attacks on CNN based malware detectors.

### 2.4.1 Proposed Solution

- **Model Architecture -** The model starts with an embedding layer, which embeds the bytes in input to a 2D matrix. The 2D matrix from the embedding layer is then fed to two Conv layers one of them using ReLU as an activation function and the second one using the sigmoid activation function. The output from both of the layers are gated, and then passed through a max-pooling layer to get a fixed size output. The output of the max-pool layer is then fed to a fully connected vanilla neural network for classification.

- **Saliency Vector -** A saliency vector is a vector in which each element/feature points out to a region of code, and the value of the element denotes the significance of that particular region i.e. The more the value the more the region is significant. [20] Chen et al and other authors

calculated the saliency vector for all the binary samples, and converted them to a matrix, and plotted a grayscale image to find various malicious and benign regions in files.

- **White-box attack methods -** White box attack means that the attacker has access to the model, knows how the model works, and what features the model takes as input.

- Appending Benign Features method works by adding cautiously selected perturbations to the malicious file. The perturbations are chosen from a benign file using the saliency vector. The attacker uses a file that is classified by the model as benign, and calculates the saliency vector for that file, and then adds benign features calculated from the saliency vector to the malicious file until it is classified as benign by the classifier.

- **Enhanced Appending Benign Feature Method -** The Enhanced Appending Benign Feature method improves the accuracy of attack by using FGSM method alongside the Appending Benign Feature method.

- **Black Box Method -** In Black-box methods, the attacker does not have access to the model, and the features used in the model.

- **Random Method -** In the Random Method, the attacker appends random data blocks from a benign file to a malicious file until the model predicts the malicious file as benign.

- **Experience-based Method -** This method relies on the experience of past successful attacks and learns what blocks serve as significant perturbations to help misclassification of malicious to benign. Although this method is highly effective, it is easy to carry out, as it requires conducting a number of successful attacks beforehand.

**Algorithm 2** The Enhanced-BFA Attack Method

**Input:** $\boldsymbol{x_0}, \boldsymbol{x}, y_{benign}, s_{filter}, L, m, \varepsilon, \beta$;
**Output:** $\tilde{\boldsymbol{x}}$;

```
A = Model(x_0);
v = GradCAM(A);
l = Index(v < 0);
q = min(p, ⌊(L − m)/s_filter⌋);
padidx = (⌊m/s_filter⌋ + 1) * s_filter;
for i = 1 to q do
    x̃ = Append(x, padidx, x_0[l_i:l_i + s_filter]);
    e_a = W(x_a)
    while J(θ, e_a, y_benign) >= β do
        g = sign(∇_{e_a} J(θ, e_a, y_benign))
        η_pad = ε Mask(g)
        e_a[padidx : padidx + i * s_filter] = Mask(e_a) − η_pad
    end while
    x̃ = W^{-1}(e_a)
    if Model.predict(x̃) == y_benign then
        return x̃;
    else
        padidx = padidx + i * s_filter;
    end if
end for
return False;
```

Figure 2.3: Enhanced Appending Benign Feature Model Architecture

### 2.4.2 Conclusion

The dataset used for performing experiments consisted of 5198 malicious files, and 5200 benign files from a clean installation of Windows. The best accuracy of the model was around 94% in classifying files as malicious or benign. The success rate of white-box attacks for FGSM was really low around 3%, whereas for BFA it was around a mean of 20%. Enhanced BFA performed really well with a mean success rate of 80%. Black blox methods did not perform that well, the random method had a mean success rate of around 4%, and the Experience based method had a mean success rate of 45%.

## 2.5 RNN based Neural Networks for Malware Detection

After the success which recurrent neural networks have shown for other tasks, they have been tried for the task of malware detection[1]. Tobiyama et al used a combination of convolutional neural networks and recurrent neural networks for the purpose of malware detection. RNNs were used for feature extraction and CNNs were used for feature classification.

### 2.5.1 Proposed Solution

- **Logging Process Behaviour -** Process monitor is used for all executed processes. The logged data of file accesses, thread creation, etc. are used as process behavior data to be fed into the RNN. The output of the RNN is used in the next step for imaging.
- **Feature Extraction and Imaging -** The output of the RNN for each log file operation is converted into a one-hot vector. These one-hot vectors are aligned to form a 2-D grayscale image.
- **Training CNN -** The greyscale image generated from the step above is used to train the CNN in the training phase with the ground truth supplied as one or zero depending on whether the original sample was malicious or not.

### 2.5.2 Conclusion

Most effective results were obtained when the image to be fed into the RNN was of size 30 X 30[3]. This proved to be an effective way to classify files as

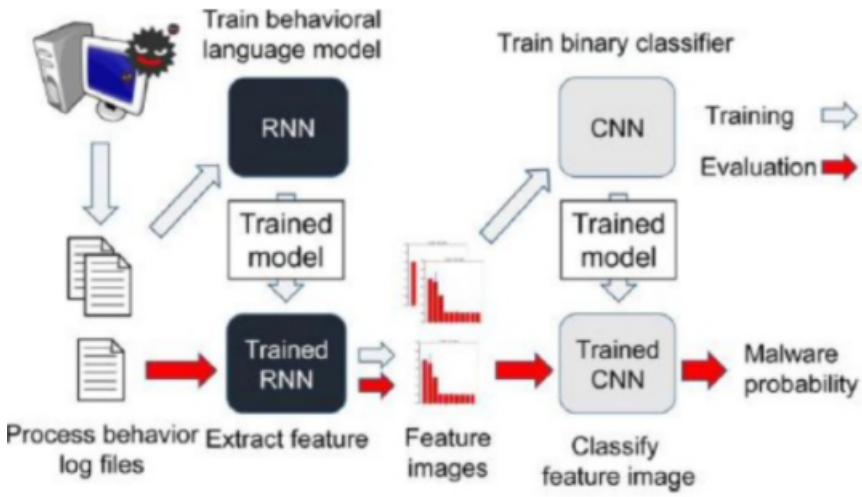


Figure 2.4: Training of greyscale image genereated using CNN

benign or malicious. Although, the robustness of the model to adversarial attacks was not checked.

## 2.6 Adversarial attacks on RNN based Malware Detector

With the use of RNN for malware detection, it became known that even they are susceptible to adversarial samples due to the general susceptibility of neural networks to adversarial attacks[4]. Generally to an attacker, the architecture of the network used for malware detection is not known and access to model weights is not present. To simulate this black-box nature of the model, Weiwei Hu et al first trained a substitute RNN to simulate the behavior of the detector to be attacked. Another RNN was trained to create adversarial samples from malware inputs. The first RNN is called Substitute RNN while the latter is known as Generative RNN[4].

Inserting irrelevant APIs into an API sequence has been the key to generating adversarial malware samples that retain their functionality. Removing APIs might break the code's functionality[4].

**Algorithm 1** Training the Proposed Model

```
while terminal condition not satisfied do
    Sample a minibatch of data, which contains malware and benign programs.
    Calculate the outputs of the generative RNN for malware.
    Get the outputs of the substitute RNN on benign programs and the Gumbel-Softmax output of malware.
    Get the outputs of the victim RNN on the adversarial examples and benign programs.
    Minimize L_S on both benign and malware data by updating the substitute RNN's weights.
    Minimize L_G on malware data by updating the generative RNN's weights.
end while
```

Figure 2.5: Generative recurrent neural network algorithm

### 2.6.1 Proposed Solution

- **Generative Recurrent Neural Network -** This neural network accepts a sequence of APIs corresponding to malware code and outputs another sequence of APIs with the same functionality but which is adversarial in nature. It does this by inserting small API sequences between each two consecutive APIs.

- **Substitute Recurrent Neural Network -** An attention-enabled two-way RNN is used to substitute the unknown RNN. Being powerful, this RNN architecture can be used to replicate the behavior of all other RNN architectures. The black-box RNN's output is used to train this substitute RNN.

- **Training -** The algorithm for the generative recurrent neural network training as shown

## 2.7 Transformer based Neural Networks for Malware Detection

As it came to light that malware detectors based on malware signatures were failing to capture novel malware, many machine learning methods were proposed as explained in previous sections[3][4]. These methods did not look at the whole meaning of the assembly code, but rather looked at different chunks of the assembly language instructions. To overcome this, transformer[5] based neural networks for malware detection were proposed by Qi et al[6]

### 2.7.1 Proposed Solution

The malware detection model I-MAD includes an H-Tran, i.e. a hierarchical Transformer to learn a vector to represent the meaning of a target executable and an IFFNN, i.e. an interpretable feed-forward neural network, that takes the vector representing the assembly code of a target executable and vectors representing other features as its inputs to produce an interpretable detection result.

- **Hierarchical Transformer -** The core of each component of H-Tran is a customized encoder of the Transformer network. The input to the Transformerencoder is a sequence of integers=⟨ x1, x2, ..., xn ⟩, and the output is a sequence of vectors=⟨ y1, y2, ..., yn ⟩.

    1. Bottom-Level Transformer to Understand Basic Blocks:The objective of the bottom-level Transformer is to generate a vector representing the semantic meaning of a basic block with the sequence of its instructions as the input.

2. Middle-Level Transformer to Understand Assembly Function: The authors have introduced the bottom-level Transformer to learn the vector representation of the semantic meaning of a basic block. They have customized the Transformer to learn the vector representation of the semantic meaning of an assembly 21function from the set of vectors representing its basic blocks.
3. Top-Level Transformer to Understand Full Logic of An Executable: With the previous two neural networks, each assembly function is represented by a vector. Next, one vector representing the full logic of an executable based on the vector representations of all its assembly functions is learned by the top-level transformer. The top-level Transformer is a duplicate of the middle-level Transformer. The top-level Transformer is fed to the IFFNN for malware detection.

- **Other Features -** When malware is packed or is polymorphic or metamorphic, the assembly code of its payload is encrypted and not statically accessible. Hence, the authors feel using only assembly code would fail to identify its malicious purpose and static analysis could still 22be useful. For this purpose, they add the following features along with the vector representations learned by the H-Tran.

  1. Printable Strings - Citing literature, the authors claim that printable strings are important because they include runtime-linked libraries and functions that are commonly used among malware, registry keys that are commonly used by malware, system paths, and sometimes the names of user-defined functions, etc.
  2. PE Imports - PE Imports are dynamically linked libraries and

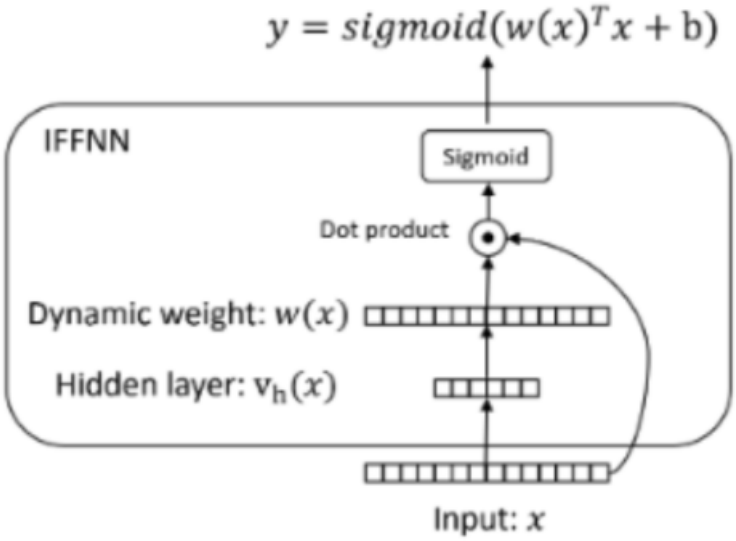


Figure 2.6: Overview of Interpretable feed-forward neural network

functions shown in the import address table of PE headers. The imports of an executable often illustrate its behaviors, e.g., modify the registry, or hook a procedure.

3. PE Header Numerical Features - There are many numerical fields in PE headers that contain information that could form different patterns among malware and benign software. Hence, the authors also use these values as features.

- **Interpretable Feed-Forward Neural Network -** Inspired by [7] which uses two attention RNN networks to form a SoftMax regression model with dynamic weights, the authors propose a novel interpretable feed-forward neural network (IFFNN) to form a "dynamic logistic regression" model. The figure below gives an overview of the IFFNN.

| Main factors leading to the prediction of the malicious class |
| --- |
| Total number of PE imports |
| Number of uncommon strings |
| Assembly code |
| The import of *comctl32.dll* |
| The frequency of the string "*\x05Shift\x0bTShiftState*" |
| Main factors leading to the prediction of the benign class |
| Assembly code |
| Number of uncommon strings |
| The import of *KERNEL32.dll* |
| Total number of PE imports |
| The frequency of the string "*\r\x01\x01\x01\x05*" |

Figure 2.7: I-MAD model interpretation

### 2.7.2 Conclusion

I-MAD achieves better accuracy than others (Mosk2008OpBi, Bald2013Meta, Saxe15Deep, Mour2019CNN) in all experiments. Additionally, the model achieves never seen before interpretability. Therefore, in addition to being a malware detector,it can also help malware analysts locate malicious payloads and find consistent patterns in malware samples.

# Chapter 3

# Problem Statement and Objectives

## 3.1 Problem Statement

Explore vulnerabilities of a transformer-based malware detector trained to classify executable files as malignant or benign by carrying out multiple adversarial attacks so that the classifier misclassifies malignant files as benign and vice versa.

## 3.2 Objectives

- To do literature review for defining research gaps.
- Collect malignant files from online repositories like VirusShare and benign files from Windows executables to generate dataset.
- Convert the dataset to be used by a machine learning model.
- Train a Transformer neural network model on the generated dataset to classify executables as benign or malignant.
- Explore vulnerabilities of this model to adversarial attacks to show that malignant files can be misclassified as benign.
- Suggest defensive mechanism against such adversarial attacks.

# Chapter 4

# Systems Requirement Specification

## 4.1 Functional Requirements

- The Transformer based neural network architecture should be capable of classifying malicious and benign files.
- The latency of the Transformer should be as minimum as possible thereby providing a almost real-time malware detection system.
- The Transformer based neural network should have high detection rate, and low false positive rate (FPR).
- Appropriate neural architecture capable of generating adversarial samples which will misclassify malignant and benign files.
- Separate defensive mechanism which will be effectively work against adversarial samples.

## 4.2 Non-Functional Requirements

- The Transformer model should be easily understandable. The required information about the model should be clearly stated.
- The system should be able to retrain on new malwares as they come.

- The code should be modular to be able to add new adversarial attack modules.
- The system design should be able to scale to real time traffic in malware detection.

## 4.3 Hardware Requirements

- NVIDIA Tesla V100 GPU
- 16 GB GPU Memory
- Intel Xeon Processor
- 64 GB RAM
- 1 TB Hard Drive
- High Speed Internet Connectivity

## 4.4 Software Requierments

- Python 3.x
- Numpy $\geq$ 3.17
- Matplotlib $\geq$ 3.1.1
- Tensorflow $\geq$ 1.14
- Tensorband $\geq$ 1.14
- CUDA for GPUs

# Chapter 5

# System Design and Architecture

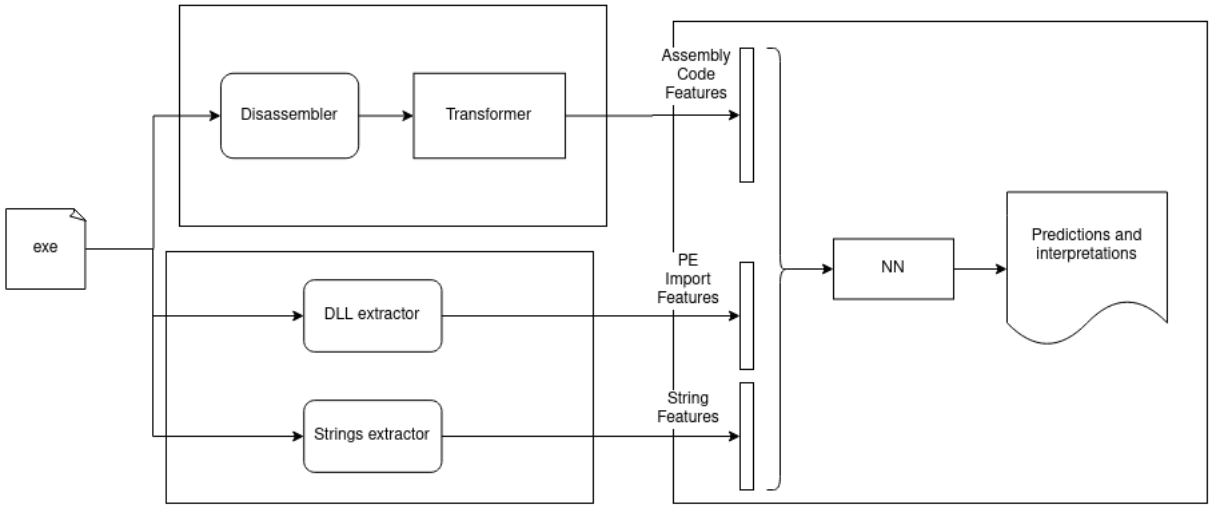


Figure 5.1: System Architecture

Our Malware detection system is divided into mainly 3 parts:

1. **Assembly Module**

   The assembly module consists of a Disassembler, a Tokenizer and a Transformer. The input to the first part is an exe file, which is fed directly to the Disassembler. The assembly module is responsible to calculate assembly language features, which would be used for final classification

2. **Static Feature Module**

   The static feature module consists of a DLL extractor, and a String extractor. The input to this is the same as that to the assembly module, an exe file. The DLL extractor extracts the PE imports from the file, and the string extractor extracts all the printable strings from the given input file. The static feature module outputs 2 set of vectors, one from each DLL extractor, and the other from string extractor. The output from the static feature module will be used for final classification.

3. **Neural Network Module**

   The neural network module consists of a neural network, which takes in assembly language features from the assembly module, and PE import features and string features from the static feature module, and performs a binary classification on whether the file is Malicious or Benign.

# Chapter 6

# Implementation and Results

## 6.1 Dataset Preparation

We performed our expermients by collecting datasets from following sources

- **VirusTotal -** A total of 2985 malware samples were collected from this source which were disassembled later.
- For collecting benign files, we used a total of 2215 executables which were a part of Windows 10 installation.

We used objdump to disassemble binary executables into .asm files. Next step is to tokenize strings and create a vocabulary which will be eventually fed to the transformer. The architecture consists of a neural network that takes the vector representing the assembly code of a target Portable Executable (PE), PE import features, string features, PE header numerical features as its input to produce interpretable detection results.

```python
import os
import subprocess
from paths import Paths as P

benign_files = os.listdir(P.benign_exe)
for idx, file in enumerate(benign_files):
    print(idx, file)
    with open(os.path.join(P.benign_exe_disassembled, file + ".asm"), "w") as outfile:
        subprocess.call(["objdump", "-Dz", "--no-show-raw-insn", "-M", "intel",
            os.path.join(P.benign_exe, file)],
            stdout=outfile)


malicious_files = os.listdir(P.malicious_exe)
for idx, file in enumerate(malicious_files):
    print(idx, file)
    with open(os.path.join(P.malicious_exe_disassembled, file + ".asm"), "w") as outfile:
        subprocess.call(["objdump", "-Dz", "--no-show-raw-insn", "-M", "intel",
            os.path.join(P.malicious_exe, file)],
            stdout=outfile)
```

## 6.2 Malware Detection Model

Our proposed system is divided into two modules :

- **Data Preparation Module :** The data preparation module is responsible for preparing data i.e. Disassembling exe files, tokenizing, creating embeddings, also extracting static features like DLLs and strings.
- **Neural Network Module :** The NN Module consists of a transformer, which takes in the embeddings from the Data Preparation module and its output is fed to the final NN, we also have a vanilla Neural Network to classify a file as malicious or benign.

### 6.2.1 Transformer Architecture

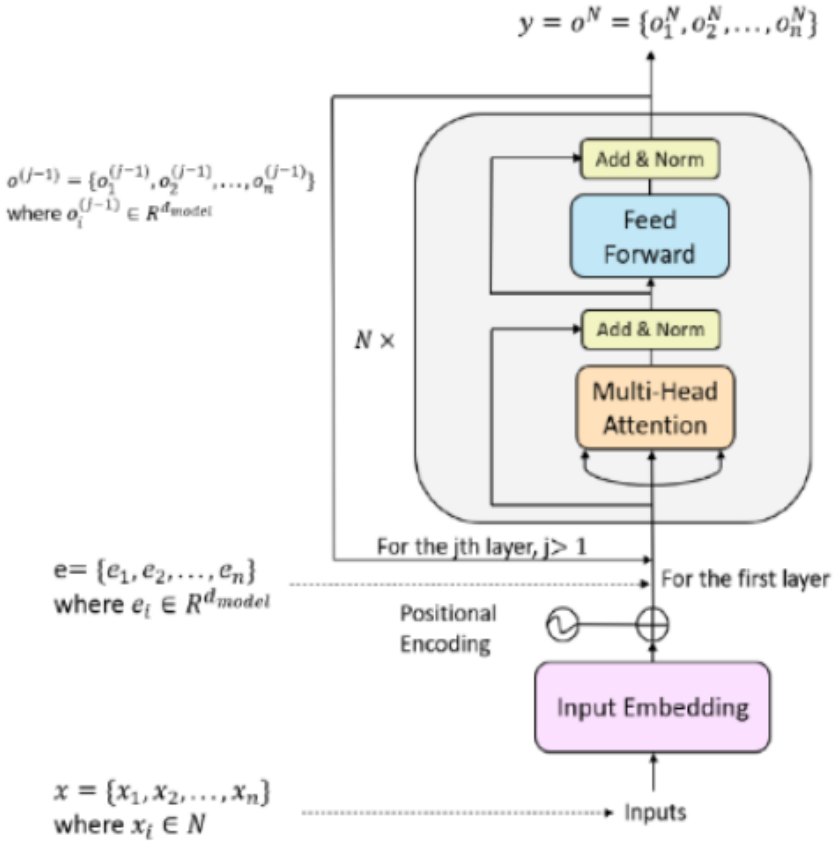


Figure 6.1: Transformer Architecture

A transformer is basically built of multiple building blocks called encoders and decoders.

- **Encoders -** The encoder basically takes in a sequence of data, and outputs a gist of all the data given as input. The encoder basically takes in a sequence of data, and outputs a gist of all the data given as input.

- **Decoders -** The decoder, being the opposite of the encoder, takes in an input of size N and produces an output of size M, where N $\leq$ M, while maintaining the focus and attention in input.

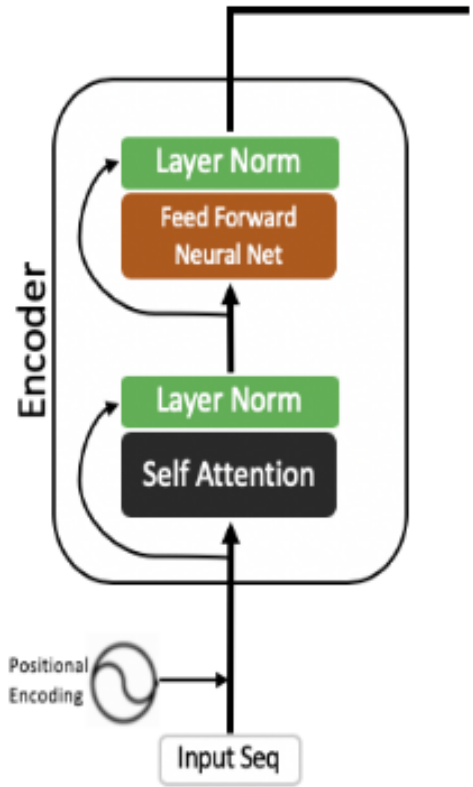


Figure 6.2: Encoder

In our model, the Encoder is a core component. The embeddings from the Data Preparation module is fed to the encoder, which then produces the gist of all the instructions fed. This serves two purposes.

- Since the output from the encoder is the gist of all the instructions, it helps in figuring out what a set of instructions is doing, in short.
- The complete instruction set cannot be used as a feature for the final vanilla Neural Network as that would be really unscalable and will have different dimensions for different input binaries.

### 6.2.2 Static Features

The DNN also takes input static features extracted from the PE binaries. The static features include :

- **PE Imports :** PE Imports are the DLL files imported by the given PE binary. This helps to capture the external function calls, and imports, and hence helps in categorizing suspicious files based on import patterns from the existing malware binaries. [J1]
- **Printable strings :** All printable strings (only ASCII characters) of size greater than or equal to 6 are extracted from the given binary and used as another feature to train the vanilla NN.

### 6.2.3 Feature Set

The final feature set for the fully connected neural network, working as a base classifier, is formed by combining the following :

- Feature vector obtained from the transformer.
- DLL feature vector obtained from Static Feature Extractor.
- String feature vector obtained from Static Feature Extractor.

The final feature vector is then fed to the fully connected neural network for classifying files as malicious or benign.

## 6.3 Adversarial attacks and Defenses

### 6.3.1 Introduction

Machine Learning techniques are generally designed to work on a specific statistical distribution. This is because, generally, the training and test data come from the same distribution of inputs. Models trained in this way are vulnerable to misclassification when provided with a set of inputs that come from a different distribution. These inputs can be modified with special techniques to force misclassification. Such generated inputs are known as adversarial examples and the fooling of a machine learning model with such adversarial examples is known as carrying out an adversarial attack.

Notable examples in the past that have successfully demonstrated adversarial attacks include McAfee's fooling of Tesla autonomous vehicle by just adding 2 black strips on a speed limit sign making the Tesla AV go 50 miles per hour past the limit.[8] Dresswear that can fool face detection systems or license plates that can fool automatic license number capture systems use adversarial examples to make such models misclassify. [9]

### 6.3.2 Classification of attacks

Adversarial attacks can be classified into two major types -

- Evasion attacks
- Poisoning attacks

An evasion attack is used when the attacker does not have access to the model during the training phase. This is the most widely used attack where input is tweaked to force the model into misclassification during the testing phase. On the other hand, a poisoning attack compromises the model in the training phase itself by giving it inputs that are malicious.

In our project, we consider evasion attacks as we consider that to be a generalized case where an attacker does not have access to the training of the model. Evasion attacks can be broadly divided into two steps -

- Estimation of sensitivity for each direction of perturbation
- Selection of directions for perturbation

In step 1, the attacker determines the amount of perturbation to add to in each direction which can most likely make the model misclassify the input. In step 2, the attacker either adds these perturbations in all directions or chooses those directions that can make the model misclassify most likely.

For the estimation of sensitivity, there are multiple algorithms available. Some of them are listed below -

- **L-BFGS -** Introduced by Christian Szegedy and others[10], this method tries to solve the minimization problem below with L-BFGS optimization method. The minimization problem is to find the minimum perturbation that can force a misclassification.

$$arg\ min_{\ r}\ f(x+r) = l \qquad (x+r) \in D \tag{6.1}$$

  where $l$ is not equal to the target label $h(x)$.

- **FGS Method -** Instead of solving the above equation with L-BFGS, Ian Goodfellow and others introduced the Fast Gradient Sign Method that can solve the equation above computationally efficiently[11].

$$X* = X + \varepsilon * sign(\nabla x C(X, Ytrue) \tag{6.2}$$

  Here $C$ is the cost function used in the model. $X*$ is the adversarial counterpart of the input $X$. $\varepsilon$ is the amount of perturbation. $\nabla x$ is the gradient of the cost function.

- **Jaccobian Method -** In an IEEE symposium paper on Cyber Security[12], Nicolas Papernot and others introduced yet another method for step 1 discussed above. They determined sensitivity in each dimension by finding out Jacobian of the trained model, i.e. it's derivative in a forward way.

### 6.3.3 Our Approach to the attack

We used the Fast Gradient Sign Method by Ian Goodfellow and others to craft our adversarial samples to fool our transformer-based malware detector. FGS Method can be implemented in two ways, either by using the target class directly or by using the iterative method. We used the target class directly to generate our adversarial samples.

$$X* = X - \varepsilon * sign(\nabla x C(X, Ytarget)) \tag{6.3}$$

Here,

$X$ = The input whose adversarial counterpart has to be generated

$X* =$ The generated adversarial sample

$Ytarget =$ The target class we want our model to misclassify this sample to

$\nabla xC =$ The derivative of the cost function with respect to the input

$\varepsilon =$ The amount of perturbation to be added to the sample.

The more the perturbation, the more are the chances that the input will be misclassified.

### 6.3.4 Results

We perturbed all possible dimensions and achieved a misclassification rate of 23.9% with the FGS Method.

### 6.3.5 Defenses against adversarial attacks

With the increase in adversarial attacks on commercial systems, many defenses have been proposed against adversarial attacks. Although these defenses do not provide complete immunity against adversarial samples, they act as deterrents. Some common and new defenses are listed below.

- **Training on adversarial samples -** One of the most common approaches for defense against adversarial attacks, in this method the model developer trains the model on adversarial examples in the training phase itself. This is a brute force approach where the input distribution of the model is expanded. Although this defense is not very useful in the case of black-box attacks as shown by Narodytska et al[13], it is still widely used as a practical defensive approach against adversarial samples.

- **Masking the gradient of the model -** Many attack methods including the FGS Method depend on the derivative of the trained

model. If models which can not be differentiated, like nearest neighbor classifiers or decision trees, are used, this can effectively deter an adversarial attack. The downside to using this defensive strategy is that an attacker can bypass this by training a substitute model that can mimic the real model's behavior and then crafting adversarial samples on the same as shown by Papernot and others[12].

- **Reducing feature space -** Developed by Xu et al[14], this method aims at reducing the feature space, also known as feature squeezing to make step 1 of generating adversarial samples discussed above difficult. With less number of features, the number of dimensions to add perturbations to declines naturally. To achieve the same effect as shown by higher dimension inputs, stronger perturbations need to be added which can break the original input's purpose.

- **Transferability block -** Adversarial attacks are successful in many cases due to the transferability property of neural networks. Transferability property says that adversarial samples generated on one neural network can make other neural networks misclassify even if they are trained on inputs that come from different data distributions and are made up of inherently different architectures. Hosseini et al[15] proposed a None method where a model is trained to classify adversarial samples into a new class called None. In our approach, we set up defenses for our model with two defenses from the ones listed above. With the most practical adversarial training, the misclassification rate dropped to 11.2%. With reducing the feature space, we did not get promising results, and the misclassification rate reduced by a mere 2.4% to 21.5%.

# Chapter 7

# Conclusion and Future Scope

The use of machine learning techniques for the task of malware detection is surely an avenue worth exploring. It has given promising results and is in use at a few of the most sought after anti-malware products[y14]. But due to the inherent nature of machine learning techniques, these malware detectors are prone to adversarial attacks. We have implemented an avant-garde machine learning detector using Google's Transformer neural network architecture, demonstrated an adversarial attack on the same, and proposed defenses against such adversarial attacks. The future scope in this domain is identified to be to demonstrate more types of adversarial attacks on such malware detectors and propose better defenses that do not need access to the trained model.

# Bibliography


[1] https://www.mcafee.com/enterprise/en-us/assets/reports/rp-quarterly-threats-aug-2019.pdf

[2] https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html

[3] S. Tobiyama, Y. Yamaguchi, H. Shimada, T. Ikuse and T. Yagi, "Malware Detection with Deep Neural Network Using Process Behavior," 2016 IEEE 40th Annual Computer Software and Applications Conference (COMPSAC), Atlanta, GA, 2016, pp. 577-582, doi: 10.1109/COMPSAC.2016.151.

[4] Black Box Based Attacks RNN based Malware Detectors, Weiwei Hu et al - May 2017

[5] Vaswani, Ashish & Shazeer, Noam & Parmar, Niki & Uszkoreit, Jakob & Jones, Llion & Gomez, Aidan & Kaiser, Lukasz & Polosukhin, Illia. (2017). Attention Is All You Need.

[6] I-MAD: A Novel Interpretable Malware Detector UsingHierarchical Transformer, Q. Li et al -Sept 2019

[7] Choi, Edward & Schuetz, Andy & Stewart, Walter & Sun, J.. (2016). Using recurrent neural network models for early detection of heart failure onset. Journal of the American Medical Informatics Association. 24. ocw112. 10.1093/jamia/ocw112.

[8] Wired News Article - https://www.wired.com/story/tesla-speed-up-adversarial-example-mgm-breach-ransomware/

[9] NYT Article - https://www.newyorker.com/magazine/2020/03/16/dressing-for-the-surveillance-age

[10] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian J. Goodfellow, and Rob Fergus.2013. Intriguing properties of neural networks.CoRRabs/1312.6199 (2013). http://arxiv.org/abs/1312.6

[11] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and Harnessing Adversarial Examples.CoRRabs/1412.6572 (2014). http://arxiv.org/abs/1412.657

[12] Nicolas Papernot, Patrick McDaniel, Somesh Jha, Matt Fredrikson, Z Berkay Celik, and Ananthram Swami. 2016. The limitations of deep learning in adversarial settings. InSecurity and Privacy (EuroS&P), 2016 IEEE European Symposium on. IEEE, 372–387.

[13] Nina Narodytska and Shiva Prasad Kasiviswanathan. 2017. Simple Black-Box Adversarial Attacks on Deep Neural Networks. In2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops, Honolulu, HI, USA, July 21-26, 2017. 1310–1318. https://doi.org/10.1109/CVPRW.2017.172

[14] Weilin Xu, David Evans, and Yanjun Qi. 2017. Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks. CoRRabs/1704.01155 (2017). arXiv:1704.01155 http://arxiv.org/abs/1704.01155

[15] Hossein Hosseini, Yize Chen, Sreeram Kannan, Baosen Zhang, and Radha Poovendran. 2017. Blocking Transferabilityof Adversarial Examples in Black-Box Learning Systems.CoRRabs/1703.04318 (2017). arXiv:1703.04318 http://arxiv.org/abs/1703.04318

[16] Kaspersky Lab Whitepaper https://media.kaspersky.com/en/enterprise-security/Kaspersky-Lab-Whitepaper-Machine-Learning.pdf

[17] J. Saxe and K. Berlin, "Deep neural network based malware detection using two dimensional binary program features," 2015 10th International Conference on Malicious and Unwanted Software (MALWARE), Fajardo, 2015, pp. 11-20, doi: 10.1109/MALWARE.2015.7413680.

[18] Attack and Defense of Dynamic Analysis-Based, Adversarial Neural Malware Classification Models. Jack W. Stokes, De Wang, Mady Marinescu, Marc Marino, Brian Bussone https://arxiv.org/abs/1712.05919

[19] M. Kalash, M. Rochan, N. Mohammed, N. D. B. Bruce, Y. Wang and F. Iqbal, "Malware Classification with Deep Convolutional Neural Networks," 2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS), Paris, 2018, pp. 1-5, doi: 10.1109/NTMS.2018.8328749.

[20] B. Chen, Z. Ren, C. Yu, I. Hussain and J. Liu, "Adversarial Examples for CNN-Based Malware Detectors," in IEEE Access, vol. 7, pp. 54360-54371, 2019, doi: 10.1109/ACCESS.2019.2913439.